

README

Everything an Examiner Needs to Access and Test the System, on One Page

Fill in the bracketed placeholders in §3 with your actual live values before submitting — see [Deployment_Guide.pdf](#) for how to get them. Everything else on this page is already accurate.

Course	CSCD602 — Advanced Software Engineering, Capstone Project
Group	Angry Nerds
Project title	FarmConnect Ghana
Document version	1.0
Date	12 August 2026
Individual submission by	George Boadu Junior — 22427354

1. Project Overview

FarmConnect Ghana is a mobile-first, bilingual (English/Twi) marketplace connecting Ghanaian smallholder farmers directly to buyers, live market prices, and an AI-powered agricultural advisor — with a full SMS command channel so feature-phone users with no data plan can participate too. Built as the CSCD602 Advanced Software Engineering capstone by Group Angry Nerds, University of Ghana.

2. Main System Features

- Phone + OTP authentication (no passwords), role-based (Farmer / Buyer / Administrator).
- Produce listings with photos, quantity, price, harvest date, and GPS location; buyer search/filter by crop, category, price, and distance.
- End-to-end order lifecycle with manual, zero-custody Mobile Money payment reconciliation and an administrator-arbitrated dispute path.
- Live, continuously-updating per-crop price feed.
- AI advisory chatbot (Google Gemini) with text + photo-based pest identification, weather-aware.
- In-app + SMS notifications for every order/admin event.
- Farmer cooperative (co-op) groups with join codes and leadership succession.
- Mutual 1–5 star ratings feeding a per-user trust score.
- Full SMS command channel (`HELP` , `REG` , `MOMO` , `LIST` , `PRICE` , `ORDERS` , `CONFIRM`) for feature-phone users.
- Admin Portal — account verification/suspension, listing moderation, dispute resolution.
- Installable, offline-capable Progressive Web App with a full bilingual and accessibility pass.

3. Technology Stack

Frontend	Vite + React 18 + TypeScript, Tailwind CSS, i18next, vite-plugin-pwa
Backend	Node.js + Express 4 + TypeScript, Prisma ORM, ioredis, Zod
Data	PostgreSQL 16, Redis 7
AI / external	Google Gemini, OpenWeatherMap, GiantSMS, Africa's Talking
Hosting	Vercel (web) · Render (API) · Supabase (Postgres) · Upstash (Redis) — all free tier

4. How to Access the Application

Live application	[FILL IN — your Vercel URL, e.g. <code>https://farmconnect-ghana.vercel.app</code>]
Admin URL	[same URL]/admin/users (log in with the seeded admin phone number first)
API base URL	[FILL IN — your Render URL, e.g. <code>https://farmconnect-api.onrender.com</code>] — /health should return <code>{"status": "ok", "db": true, "redis": true}</code>
Source code	[FILL IN — your GitHub repository URL]

5. Test Credentials

There are no fixed passwords anywhere in this system — authentication is phone number + a one-time SMS code. To test as a Farmer or Buyer:

1. Open the live application URL.
2. Enter any Ghanaian-format phone number (e.g. `024xxxxxxx`) not already registered.
3. Retrieve the OTP from the SMS sent to that number — **or**, if GiantSMS credentials are not configured on the deployment, the OTP is logged in the Render service's log output instead of being sent (see `Deployment_Guide.pdf` Step 4 to configure real SMS sending).
4. Choose Farmer or Buyer on first login.

6. Admin Access

There is no self-service admin sign-up. An administrator account must be seeded once against the production database:

```
DATABASE_URL="<production connection string>" \
  npx tsx apps/api/prisma/seed.ts --phone [FILL IN ADMIN PHONE] --name "[FILL IN ADMIN NAME]"
```

That phone number then logs in through the same phone+OTP screen as everyone else and lands in the Admin Portal.

7. Deployment Information

Deployed on an entirely free-tier stack: Vercel (static PWA), Render (API, free web service — sleeps after ~15 min idle, ~30–50s cold start on first request after), Supabase (managed Postgres 16), Upstash (managed Redis). Both Vercel and Render redeploy automatically on every push to `main`. Full step-by-step deployment instructions are in `Deployment_Guide.pdf`.

8. Known Limitations

- Payments are manually reconciled, not automated escrow (see SRS §9.1).
- Market prices are a realistic simulation, not a real Ghana Commodity Exchange feed (see SRS §9.2).
- 10,000-concurrent-user and 99.5%-uptime targets are architecturally plausible but not load-tested at this submission's scale (see Testing_Report.pdf §11).
- Render's free tier cold-starts after idling — the first request after a quiet period may take 30–50 seconds.

9. Special Instructions for Testing

- To test the AI advisor, a `GEMINI_API_KEY` must be configured on the Render service (Deployment_Guide.pdf Step 4) — without it, the advisor deliberately fails with a clear 503 rather than fabricating a response.
- To test the full farmer flow, Mobile Money details must be linked first (Profile → Mobile Money) — listing creation is blocked until this is done, by design.
- The full end-to-end test evidence already captured for this build (69 automated tests + 11 manual smoke tests, all passing) is in Testing_Report.pdf — useful context if any behaviour looks unexpected during live testing.